



SSC502 – Laboratório de ICC I

Controle de Fluxo em C

Prof.Dr. Danilo Spatti

- O comando `if` representa uma tomada de decisão do tipo "**SE isto ENTÃO aquilo**". A sua forma geral é:

```
if (condição) declaração;
```

- A **condição** do comando `if` é uma **expressão** que será avaliada.

- Se o resultado for **zero** a declaração **não** será **executada**.
- Se o resultado for **qualquer** coisa **diferente** de zero, a **declaração** será **executada**.
- A **declaração** pode ser um **bloco** de código ou apenas **um comando**.

```
1  # include <stdio.h>
2
3  int main()
4  {
5      int num;
6      printf("Digite um numero:");
7      scanf("%d",&num);
8      if (num > 10) printf("\nNumero maior que 10");
9      if (num == 10)
10     {
11         printf("\nAcertou!");
12         printf("\nNumero igual a 10");
13     }
14     if (num < 10) printf("\nNumero menor que 10");
15     return 0;
16 }
```

Digite um numero:12

Numero maior que 10

Process returned 0 (0x0) execution time : 1.929 s

Press any key to continue.

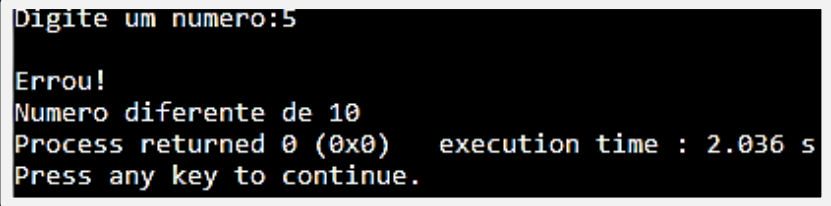
- No programa anterior a expressão `num > 10` é avaliada e retorna um valor diferente de 0, se verdadeira, e 0, se falsa.
- No exemplo, se `num` for maior que 10, será impressa a frase: "Numero maior que 10".
- Repare que, se o número for igual a 10, estamos executando dois comandos.

- Para que isto fosse possível, tivemos que agrupá-los em um bloco que se inicia logo após a comparação e termina após o segundo `printf`.
- Repare também que quando queremos testar igualdades usamos o operador `==` e não `=`.
- Isto porque o operador `=` representa apenas uma atribuição.

- Podemos pensar no comando `else` como sendo um **complemento** do comando `if`.
- O comando `if` **completo** tem a seguinte forma geral:

```
if (condição) declaração_1;  
else declaração_2;
```

```
1 # include <stdio.h>
2
3 int main()
4 {
5     int num;
6     printf("Digite um numero:");
7     scanf("%d",&num);
8     if (num == 10)
9     {
10         printf("\nAcertou!");
11         printf("\nNumero igual a 10");
12     }
13     else
14     {
15         printf("\nErrou!");
16         printf("\nNumero diferente de 10");
17     }
18     return 0;
19 }
```



Digite um numero:5
Errou!
Numero diferente de 10
Process returned 0 (0x0) execution time : 2.036 s
Press any key to continue.

- A estrutura `if else if` é apenas uma **extensão** da **estrutura** `if else`.

```
if (condição_1) declaração_1;  
else if (condição_2) declaração_2;  
else if (condição_3) declaração_3;  
(...)  
else if (condição_n) declaração_n;  
else declaração_padrão;
```

- O programa **começa** a testar as **condições** começando **pela 1** e continua a **testar** até que **ele** ache uma **expressão** cujo **resultado** dê **diferente** de 0.
- Neste caso ele **executa** a **declaração** correspondente.
- Só uma **declaração** será executada, ou seja, só será executada a **declaração equivalente** à **primeira** condição que der **diferente** de zero.
- A última declaração (padrão) é a que será **executada** no caso de **todas** as **condições** darem 0 e é **opcional**.

```
1  # include <stdio.h>
2
3  int main()
4  = {
5      int num;
6      printf("Digite um numero:");
7      scanf("%d",&num);
8      if (num > 10) printf("\nNumero maior que 10");
9      else if (num == 10)
10     = {
11         printf("\nAcertou!");
12         printf("\nNumero igual a 10");
13     }
14     else if (num < 10) printf("\nNumero menor que 10");
15     return 0;
16 }
```

Digite um numero:11

Numero maior que 10

Process returned 0 (0x0) execution time : 1.757 s

Press any key to continue.

- O `if aninhado` é simplesmente um `if` dentro da declaração de um outro `if` externo.
- O único cuidado que devemos ter é o de saber exatamente a qual `if` um determinado `else` está ligado.

```
1  # include <stdio.h>
2
3  int main()
4  {
5      int num;
6      printf("Digite um numero:");
7      scanf("%d",&num);
8      if (num == 10)
9      {
10         printf("\nAcertou!");
11         printf("\nNumero igual a 10");
12     }
13 }
14 else
15 {
16     if (num > 10)
17     {
18         printf("\nNumero maior que 10");
19     }
20     else
21     {
22         printf("\nNumero menor que 10");
23     }
24 }
25 return 0;
26 }
```

Digite um numero:11

Numero maior que 10

Process returned 0 (0x0) execution time : 1.283 s

Press any key to continue.

- A estrutura if-else e o comando switch são os dois **comandos** de tomada de **decisão**.
- Sem dúvida alguma o mais **importante** dos dois é o **if**, mas o comando switch tem aplicações **valiosas**.
- Mais uma vez vale lembrar que devemos usar o **comando certo** no **local certo**.

- Isto assegura um código **limpo** e de fácil **entendimento**.
- O comando switch é **próprio** para se **testar** uma **variável** em relação a diversos **valores** pré-estabelecidos.
- Podemos fazer uma **analogia** entre o switch e a estrutura if-else-if apresentada **anteriormente**.
- A diferença fundamental é que a estrutura switch **não** aceita **expressões**. Aceita **apenas constantes**.

```
switch (variável)
{
    case constante_1:
        declaração_1;
        break;
    case constante_2:
        declaração_2;
        break;
    (...)
    default:
        declaração_default;
}
```


- O switch testa a **variável** e executa a **declaração** cujo **case** corresponda ao valor **atual** da **variável**.
- A declaração default é **opcional** e será executada apenas se a variável, que está sendo testada, **não** for **igual** a nenhuma das **constantes**.
- O comando break, faz com que o switch seja **interrompido** assim que uma das **declarações** seja executada.

```
1  # include <stdio.h>
2
3  int main()
4  {
5      int num;
6      printf("Digite um numero:");
7      scanf("%d",&num);
8      switch (num)
9      {
10         case 9:
11             printf("\nNumero igual a 9");
12             break;
13         case 10:
14             printf("\nNumero igual a 10");
15             break;
16         case 11:
17             printf("\nNumero igual a 11");
18             break;
19         default:
20             printf("\nNumero nao e nem 9, nem 10, nem 11");
21             break;
22     }
23     return 0;
24 }
```

Digite um numero:10

Numero igual a 10

Process returned 0 (0x0) execution time : 4.198 s

Press any key to continue.

- for é a **primeira** de uma série de **três estruturas** para se trabalhar com **loops** de **repetição**.
- As outras são **while** e **do**.
- As três compõem a **segunda** família de comandos de **controle** de **fluxo**.
- Podemos pensar nesta **família** como sendo a das **estruturas** de **repetição controlada**.

- Como já foi dito, o loop **for** é usado para **repetir** um **comando**, ou **bloco** de **comandos**, diversas vezes, de maneira que se possa ter um bom controle sobre o loop.

```
for (inicialização; condição; incremento) declaração;
```

- Podemos ver, então, que o for executa a **iniciação incondicionalmente** e testa a **condição**.
- Se a condição for **falsa** ele **não faz** mais nada.
- Se a condição for **verdadeira** ele **executa** a **declaração**, faz o **incremento** e volta a **testar** a condição.

- Ele fica **repetindo** estas **operações** até que a condição seja **falsa**.
- Um ponto importante é que **podemos omitir** qualquer um dos elementos do **for**, isto é, se não quisermos uma **iniciação** poderemos **omiti-la**.

```
1 # include <stdio.h>
2
3 int main()
4 {
5     int i;
6     for (i = 1; i <= 100; i++) printf("%d ", i);
7     return 0;
8 }
```

```
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27
28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49 50 5
1 52 53 54 55 56 57 58 59 60 61 62 63 64 65 66 67 68 69 70 71 72 73 74
75 76 77 78 79 80 81 82 83 84 85 86 87 88 89 90 91 92 93 94 95 96 97 98
99 100
Process returned 0 (0x0)   execution time : 0.016 s
Press any key to continue.
```

- Este loop chama-se **loop infinito** porque será **executado** para **sempre** (não existindo a condição, ela será sempre considerada verdadeira), a não ser que ele seja **interrompido**.
- Para **interromper** um loop como este usamos o comando **break**.

```
for (;;) declaração;
```



```
1  # include <stdio.h>
2
3  int main()
4  {
5      char ch;
6      printf("\nDigite uma letra - x para sair ");
7      for (;;)
8      {
9          ch = getch();
10         printf("\nLetra: %c \n",ch);
11         if (ch == 'x') break;
12     }
13     return 0;
14 }
```

```
Digite uma letra - x para sair
Letra: a
```

```
Letra: b
```

```
Letra: c
```

```
Letra: 3
```

```
Letra: x
```

```
Process returned 0 (0x0)   execution time : 6.865 s
Press any key to continue.
```

```
while (condição) declaração;
```

- Podemos ver que a estrutura while **testa** uma **condição**.
- Se esta for **verdadeira** a declaração é executada e faz-se o teste **novamente**, e assim por diante.
- Assim como no caso do for, podemos fazer um **loop infinito**.

- Para tanto basta colocar uma **expressão eternamente** verdadeira na condição.
- O programa a seguir é executado **enquanto** i for **menor** que 100.

```
1  # include <stdio.h>
2
3  int main()
4  {
5      int i = 0;
6      while (i < 100) printf(" %d", i++);
7      return 0;
8  }
```

```
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25
26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48
49 50 51 52 53 54 55 56 57 58 59 60 61 62 63 64 65 66 67 68 69 70 71
72 73 74 75 76 77 78 79 80 81 82 83 84 85 86 87 88 89 90 91 92 93 94
95 96 97 98 99
Process returned 0 (0x0)   execution time : 0.016 s
Press any key to continue.
```

```
1  # include <stdio.h>
2
3  int main()
4  {
5      char ch;
6      ch = '\0';
7      while (ch != 'q') scanf ("%c", &ch);
8      return 0;
9  }
```

x
y
q

Process returned 0 (0x0) execution time : 9.000 s
Press any key to continue.

```
do  
{  
    declaração;  
}while (condição);
```

- Mesmo que a declaração **seja apenas** um **comando** é uma boa **prática** deixar as **chaves**.
- O ponto-e-vírgula final é **obrigatório**.

- Vemos pela análise do **bloco** anterior que a estrutura do-while **executa** a **declaração**, testa a **condição** e, se esta for **verdadeira**, volta para a **declaração**.
- A grande novidade no comando do-while é que ele, ao **contrário** do for e do while, **garante** que a **declaração** será **executada** pelo **menos uma** vez.
- Um dos usos da estrutura do-while é em **menus**, nos quais você **quer garantir** que o valor **digitado** pelo usuário seja **válido**.

```
1 # include <stdio.h>
2
3 int main()
4 {
5     int i;
6     do
7     {
8         printf("\nEscolha pelo numero: ");
9         printf("\n(1)...Steinhager");
10        printf("\n(2)...Stolichnaya");
11        printf("\n(3)...Corote");
12        printf("\n");
13        scanf ("%d",&i);
14    } while ( (i < 1) || (i > 3) );
15    switch(i)
16    {
17        case 1:
18            printf("\nVoce escolheu Steinhager");
19            break;
20        case 2:
21            printf("\nVoce escolheu Stolichnaya");
22            break;
23        case 3:
24            printf("\nVoce escolheu Corote");
25            break;
26    }
27    return 0;
28 }
```

Escolha pelo numero:

(1)...Steinhager

(2)...Stolichnaya

(3)...Corote

2

Voce escolheu Stolichnaya

Process returned 0 (0x0) execution time : 1.789 s

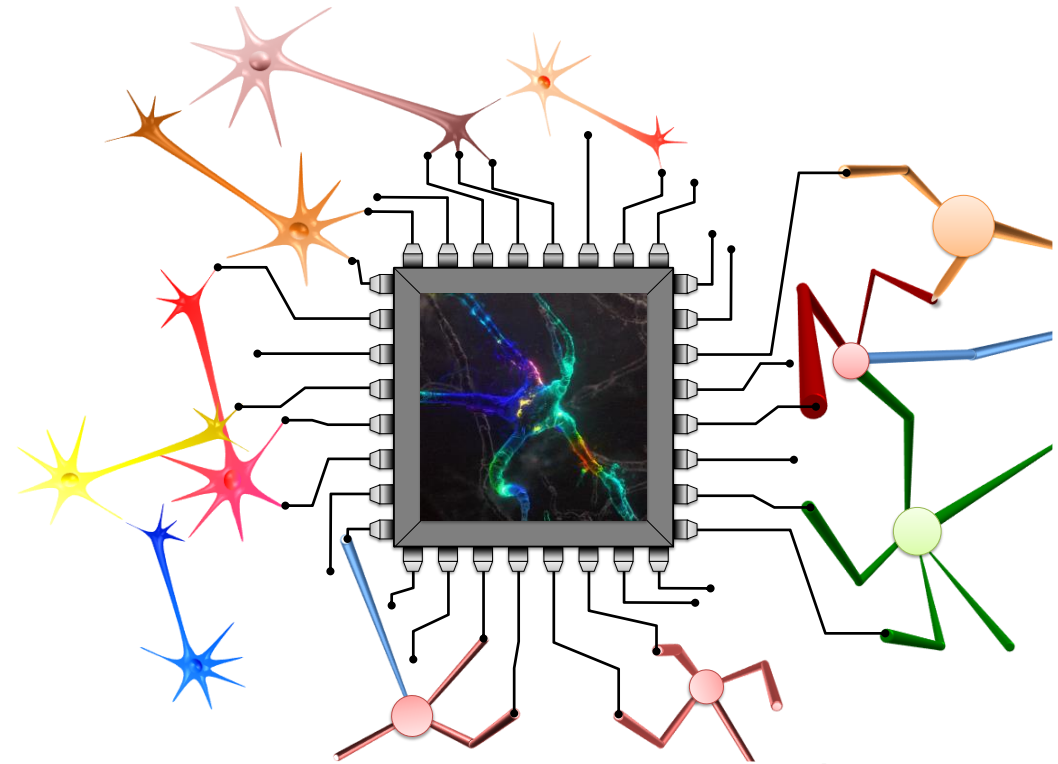
Press any key to continue.

- Escreva um programa que pede para o usuário entrar um número correspondente a um dia da semana e que então apresente na tela o nome do dia.
- Faça o programa utilizando o comando if-else e o comando switch.

- Faça um programa que apresente na tela os 100 primeiros números ímpares.
- Utilize for.

- Crie um programa que apresente ao usuário um menu com as opções de operações matemáticas abaixo.
- Receba os números e apresente os resultados correspondentes caso necessário. Utilize função para ficar modular.
 - 1 – Fatorial
 - 2 – Divisível por 3
 - 3 – Divisível por 2
 - 4 - Sair

spatti@icmc.usp.br



GE4Bio – Grupo de Estudos em Sinais Biológicos